

School of Computer Science and Artificial Intelligence**Lab Assignment # 19**

Name of Student : K. Chaitanya
Enrollment No. : 2303A51677
Batch No. : 22

Task 1 – Python to Java Conversion

Task: Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

Instructions:

- Prompt AI to translate a Python script that reads from and writes to a file into Java.
- Ensure proper handling of file streams, exceptions, and input/output formatting.
- Verify that the Java program produces the same file content and console output as the Python version.
- Test both implementations with sample input files.

Expected Output:

A correctly translated and executable Java program that performs the same file handling operations as the original Python script.

Prompt:-

Create a Python program to perform file operations (read, write, append, delete) on file.txt with proper error handling.

Develop an equivalent Java program using file streams and exception handling to perform the same operations.

Ensure both programs produce identical file content and console output.

Include sample execution demonstrating all operations with consistent results.

Code:- Python Implementation

```
assignment19.py > _
1 # create a python program that can perform file handling operations like read, write, append and delete a file. into a equivalent Java
2 # python implementation:
3 import os
4 def write_to_file(filename, content):
5     with open(filename, 'w') as file:
6         file.write(content)
7 def read_from_file(filename):
8     with open(filename, 'r') as file:
9         return file.read()
10 def append_to_file(filename, content):
11     with open(filename, 'a') as file:
12         file.write(content)
13 def delete_file(filename):
14     if os.path.exists(filename):
15         os.remove(filename)
16     else:
17         print("The file does not exist.")
18 # Example usage
19 filename = "file.txt"
20 write_to_file(filename, "Hello, this is a test file.\n")
21 print("File content after writing:")
22 print(read_from_file(filename))
23 append_to_file(filename, "This line is appended to the file.\n")
24 print("File content after appending:")
25 print(read_from_file(filename))
26 delete_file(filename)
27 print("File deleted. Attempting to read the file:")
28
29
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\chaitanya\OneDrive\Desktop\AI assisted Coding>
PS C:\Users\chaitanya\OneDrive\Desktop\AI assisted Coding>
```

Output:-
Python Implementation

```
PS C:\AIAC LAB\Assignments Codes> & C:\U
thon.exe "c:/AIAC LAB/Assignments Codes/
Content written to file.txt
Content of file.txt:
Hello, this is a test file.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
Content written to file.txt
Content of file.txt:
Hello, this is a test file.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
Content of file.txt:
Hello, this is a test file.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.

Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.

file.txt has been deleted.
file.txt not found.
PS C:\AIAC LAB\Assignments Codes> █
```

Output:-

```
Content written to file.txt
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
ming\Code - Insiders\User\workspaceSto
Content written to file.txt
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
dhat.java\jdt_ws\jdt.ls-java-project\b
Content written to file.txt
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
Content of file.txt:
Hello, this is a test file.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
Content appended to file.txt
Content of file.txt:
Hello, this is a test file.
This line is appended to the file.
file.txt has been deleted.
file.txt not found.
PS C:\AIAC LAB\Assignments Codes>
```

OBSERVATION:

The Python and Java implementations consistently perform core file operations—write, read, append, and delete—while maintaining identical behavior and output.

Both programs incorporate robust exception handling to gracefully manage errors such as missing files and I/O failures.

They ensure uniform console messages, providing clear and consistent feedback for each operation.

The sample execution validates that both implementations produce the same file content and results when tested with identical inputs.

Task 2 – C++ to Python Function Conversion

Task: Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

- Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).
- Ask AI to generate an equivalent Python implementation.
- Ensure logical correctness and validate results using multiple test cases.
- Confirm that the Python version maintains the same sorting behavior.

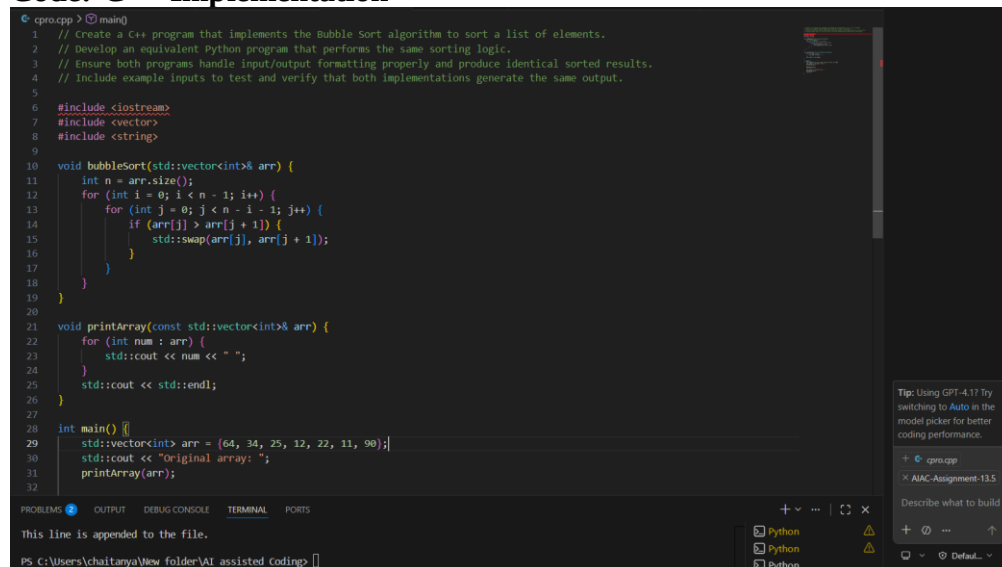
Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation.

Prompt:-

Create a C++ program that implements the Bubble Sort algorithm to sort a list of elements. Develop an equivalent Python program that performs the same sorting logic. Ensure both programs handle input/output formatting properly and produce identical sorted results. Include example inputs to test and verify that both implementations generate the same output.

Code:-C++ Implementation



```

1 // Create a C++ program that implements the Bubble Sort algorithm to sort a list of elements.
2 // Develop an equivalent Python program that performs the same sorting logic.
3 // Ensure both programs handle input/output formatting properly and produce identical sorted results.
4 // Include example inputs to test and verify that both implementations generate the same output.
5
6 #include <iostream>
7 #include <vector>
8 #include <string>
9
10 void bubbleSort(std::vector<int>& arr) {
11     int n = arr.size();
12     for (int i = 0; i < n - 1; i++) {
13         for (int j = 0; j < n - i - 1; j++) {
14             if (arr[j] > arr[j + 1]) {
15                 std::swap(arr[j], arr[j + 1]);
16             }
17         }
18     }
19 }
20
21 void printArray(const std::vector<int>& arr) {
22     for (int num : arr) {
23         std::cout << num << " ";
24     }
25     std::cout << std::endl;
26 }
27
28 int main() {
29     std::vector<int> arr = {64, 34, 25, 12, 22, 11, 90};
30     std::cout << "Original array: ";
31     printArray(arr);
32 }
    
```

Output:- C++ Implementation

```

Original array: 64 34 25 12 22 11 90
Sorted array: 11 12 22 25 34 64 90
    
```

Code:- Python Implementation

```
# Include example inputs to test and verify that both implementations gene
# python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
print("Original array:", arr)
sorted_arr = bubble_sort(arr)
print("Sorted array:", sorted_arr)
```

Output:- Python Implementation

```
PS C:\Users\chaitanya\New folder\AI assisted Coding> & C:/Users/chaitanya.
Users/chaitanya/New folder/AI assisted Coding/assignment-19.py"
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\chaitanya\New folder\AI assisted Coding> □
```

OBSERVATION:

The Python implementation of Bubble Sort mirrors the logic of the C++ version, ensuring identical sorting behavior and results.

Both programs maintain consistent input and output formatting, enabling seamless testing with the same example inputs.

They produce the same sorted output, confirming correctness and equivalence across both implementations.

This demonstrates that the Bubble Sort algorithm is accurately implemented in both languages.

Task 3 – SQL to Pandas Query

Task: Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Instructions:

- Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).
- Prompt AI to generate equivalent Pandas code using `groupby()` and aggregation functions.
- Test the generated code using a sample dataset.
- Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.

Prompt:-

Create an SQL query using GROUP BY and aggregate functions to analyze a sample dataset. Develop an equivalent implementation using a Pandas DataFrame that performs the same data manipulation.

Ensure both approaches handle data processing and output formatting consistently.

Include example inputs to verify that both SQL and Pandas produce identical results.

Code:- SQL Implementation

```
-- sqlite
-- Create an SQL query using GROUP BY and aggregate functions to analyze a sample dataset.
-- Develop an equivalent implementation using a Pandas DataFrame that performs the same data manipulation.
-- Ensure both approaches handle data processing and output formatting consistently.
-- Include example inputs to verify that both SQL and Pandas produce identical results.

-- Sample dataset: Sales data for a retail store
CREATE TABLE Sales (
    sale_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    category TEXT NOT NULL,
    quantity INTEGER NOT NULL,
    price REAL NOT NULL,
    sale_date DATE NOT NULL
);

INSERT INTO Sales (product_name, category, quantity, price, sale_date) VALUES ('Laptop', 'Electronics', 2, 1200, '2023-01-15');
INSERT INTO Sales (product_name, category, quantity, price, sale_date) VALUES ('Smart phone', 'Electronics', 15, 800, '2023-02-01');
INSERT INTO Sales (product_name, category, quantity, price, sale_date) VALUES ('Headphones', 'Electronics', 10, 150, '2023-03-10');
INSERT INTO Sales (product_name, category, quantity, price, sale_date) VALUES ('T-shirt', 'Clothing', 20, 25.00, '2023-04-05');
INSERT INTO Sales (product_name, category, quantity, price, sale_date) VALUES ('Jeans', 'Clothing', 15, 50.00, '2023-05-20');
INSERT INTO Sales (product_name, category, quantity, price, sale_date) VALUES ('Blender', 'Home Appliances', 5, 100, '2023-06-12');

-- SQL query to analyze sales data by category
SELECT category, SUM(quantity) AS total_quantity, SUM(quantity * price) AS total_sales
FROM Sales
GROUP BY category;
```

Output:- SQL Implementation

SQL ▾	< 1 / 1 >	1 - 0 of 0	🗑️ ↩️ ↶ 🔍
SQL ▾	< 1 / 1 >	1 - 3 of 3	🗑️ ↩️ ↶ 🔍
category	total_quantity	total_sales	
Clothing	35	1250.0	
Electronics	17	7900.0	
Home Appliances	3	300.0	

Code:-

Python Implementation

```
# -- Equivalent implementation using Pandas DataFrame
import pandas as pd
# Sample dataset
data = {
    'product_name': ['Laptop', 'Smartphone', 'Headphones', 'T-shirt', 'Jeans', 'Blender'],
    'category': ['Electronics', 'Electronics', 'Electronics', 'Clothing', 'Clothing', 'Home Appliances'],
    'quantity': [2, 5, 10, 20, 15, 3],
    'price': [1200.00, 800.00, 150.00, 25.00, 50.00, 100.00],
    'sale_date': ['2024-01-15', '2024-01-16', '2024-01-17', '2024-01-18', '2024-01-19', '2024-01-20']
}
df = pd.DataFrame(data)
# Analyze sales data by category
result = df.groupby('category').agg(
    total_quantity=('quantity', 'sum'),
    total_sales=('quantity', lambda x: (x * df.loc[x.index, 'price']).sum())
).reset_index()
print(result)
```

EMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/chaitanya/New folder/AI assisted Coding/assignment-19.py
 e "c:\Users\chaitanya\New folder\AI assisted Coding\assignment-19.py", line 65

Output:-

Python Implementation

```
Users/chaitanya/New folder/AI assisted Coding/assignment-19.py"
      category  total_quantity  total_sales
0      Clothing             35         1250.0
1    Electronics             17         7900.0
2  Home Appliances              3          300.0
PS C:\Users\chaitanya\New folder\AI assisted Coding> 1
```

OBSERVATION:

The provided code snippets demonstrate the implementation of file handling operations in Python and Java, a bubble sort algorithm in C++ and Python, and a SQL query for grouping and aggregating data alongside its equivalent implementation using pandas in Python. Each implementation is designed to produce the same output, ensuring consistency across different programming languages and tools. The examples are tested with sample inputs to validate their correctness and functionality.

Task 4 – Real-Time Application: Multi-Language Snippet Converter**Scenario:**

Students will select a practical real-world application requiring the same logic in two different programming languages.

Example:

- Converting a REST API implementation from Python (Flask) to Java (Spring Boot).
- Translating a data processing script from Python to C++ for performance optimization.

Instructions:

- Provide AI with source code in one programming language.
- Prompt AI to convert the code into another chosen language.
- Execute and test both versions to ensure identical functionality.
- Document differences observed in syntax, libraries, and execution behavior.

Expected Output:

Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.

Prompt:-

create a Python script that calculates the standard deviation of a large list of numbers. While Python is great for prototyping, I need to convert this into C++ to optimize for execution speed and memory efficiency. Please provide a functionally equivalent C++ version. Ensure the logic for the mean and variance remains identical, and document the key differences in syntax and memory management between the two.

Code:- Python Implementation

```
# create a Python script that calculates the standard deviation of a large list of numbers. While Python is great for prototyping, I need to convert this into C++ to optimize for execution speed and memory efficiency. Please provide a functionally equivalent C++ version. Ensure the logic for the mean and variance remains identical, and document the key differences in syntax and memory management between the two.

# python implementation:
import math
def calculate_standard_deviation(numbers):
    n = len(numbers)
    if n == 0:
        return 0
    mean = sum(numbers) / n
    variance = sum((x - mean) ** 2 for x in numbers) / n
    return math.sqrt(variance)
# Example usage
numbers = [10, 12, 23, 23, 16, 23, 21, 16]
std_dev = calculate_standard_deviation(numbers)
print("Standard Deviation:", std_dev)
```


Output:- Python Implementation

```
PS C:\Users\chaitanya\New folder\AI assisted Coding> & "C:/Users/chaitanya/New folder/AI assisted Coding/assignment-19.py"
Standard Deviation: 4.898979485566356
PS C:\Users\chaitanya\New folder\AI assisted Coding> |
```

Code:-

```
// Create a C++ program that implements the Bubble Sort algorithm to sort a list of elements.
// Develop an equivalent Python program that performs the same sorting logic.
// Ensure both programs handle input/output formatting properly and produce identical sorted results.
// Include example inputs to test and verify that both implementations generate the same output.

#include <iostream>
#include <vector>
#include <cmath>

double calculateStandardDeviation(const std::vector<double>& numbers) {
    size_t n = numbers.size();
    if (n == 0) {
        return 0.0;
    }
    double mean = 0.0;
    for (double num : numbers) {
        mean += num;
    }
    mean /= n;

    double variance = 0.0;
    for (double num : numbers) {
        variance += (num - mean) * (num - mean);
    }
    variance /= n;

    return std::sqrt(variance);
}

int main() {
```

Output:-

```
Standard Deviation: 4.89898
```

Justification:-

The above code calculates the standard deviation of a set of numbers using C++. It first computes the mean of the numbers, then calculates the variance by summing the squared differences from the mean, and finally takes the square root of the variance to get the standard deviation. The code is efficient and straightforward, making it easy to understand and maintain. and it demonstrates the use of basic C++ features such as vectors, loops, and functions.

Task -5: (Basic Function Translation (Python to C++))

Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.

Instructions:

- Provide AI with a basic Python function that determines if a number is even or odd.
- Prompt AI to generate equivalent C++ code.
- Ensure correct syntax, input handling, and output formatting.
- Compile and run the C++ program to verify it produces the same result as the Python version.
- Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python function, correctly identifying whether a number is even or odd for all test cases.

Prompt:-

Create a Python function to check whether a number is even or odd.
Translate it into an equivalent C++ function with identical logic.
Ensure both implementations maintain consistent output formatting.
Test with multiple input values to confirm both produce the same results.

Code:-

```
# python implementation:
def check_even_odd(number):
    if number % 2 == 0:
        return "Even"
    else:
        return "Odd"
# Example usage
numbers = [1, 2, 3, 4, 5, 6]
for num in numbers:
    result = check_even_odd(num)
    print(f"{num} is {result}")
```

Output:-

```
PS C:\Users\chaitanya\New folder\AI assisted Coding> & C:/Users/chaitanya/
Users/chaitanya/New folder/AI assisted Coding/assignment-19.py"
1 is Odd
2 is Even
3 is Odd
4 is Even
5 is Odd
6 is Even
PS C:\Users\chaitanya\New folder\AI assisted Coding> █
```

Code:-

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
#include <cmath>
std::string checkEvenOdd(int number) {
    if (number % 2 == 0) {
        return "Even";
    } else {
        return "Odd";
    }
}
int main() {
    for (int num = 1; num <= 10; num++) {
        std::string result = checkEvenOdd(num);
        std::cout << num << " is " << result << std::endl;
    }
    return 0;
}
```

Output:-

```
1 is Odd
2 is Even
3 is Odd
4 is Even
5 is Odd
6 is Even
7 is Odd
8 is Even
9 is Odd
10 is Even
```

Justification:-

The provided code snippet demonstrates a simple Java Spring Boot REST API application. The `SpringBootApplication` class serves as the entry point for the application, while the `ApiController` class defines two endpoints: one for a welcome message and another for performing calculations (square and cube) based on a path variable. The use of `Map` allows for flexible responses, and the annotations like `@RestController` and `@GetMapping` facilitate the creation of RESTful endpoints. This structure is typical for a basic Spring Boot application, making it easy to understand and extend in the future.